

 Week 10: Why RAG Breaks & What Comes Next

Context Rot, Production Reality, and the Evolution of Retrieval Systems

WEEK

10

DATE

23 MAR 2026

FOCUS

PRODUCTION RAG

Your RAG works on demos. It breaks on production data. The "just add more context" era is over—Naive RAG looks magical on clean, short documents but catastrophically degrades on real-world data due to Context Rot and distractor interference. This week, we tear down the "Top-K" illusion, reverse-engineer how ChatGPT actually manages memory (spoiler: no vector databases), and build surgical precision into our pipelines using Hybrid Search, Reranking, HyDE, and the vectorless frontier of PageIndex.

 Quick Links & Resources

Resource	Link
 Lecture Slides	Google Drive
 Colab Notebook	Google Colab (Local Copy: Codes/cLass_7b.ipynb)
 Excalidraw Board	Interactive Whiteboard
 Assignment	Google Colab — Last Cell

 Table of Contents

#	Section	Core Concept
	The Core Thesis	The promise vs. the reality of large context windows.
1	Context Rot: The Hidden Killer	Why "just add more context" degrades your outputs.
2	The Distractors Problem	Nicely formatted documents might actually hurt retrieval.
3	Naive RAG vs Production	Why weekend demos work but enterprise systems fail.
4	How ChatGPT Manages Memory	Reverse-engineering their surgical (non-vector) approach.
5	Advanced RAG Techniques	The modern 3-pillar paradigm: Retrieval, Chunking, Generation.
6	Deep Dive: Hybrid Search	Combining BM25 keywords with Semantic Vectors via RRF.
7	Deep Dive: Reranking	The Two-Pass system for absolute precision scoring.
8	Deep Dive: HyDE	Bridging the semantic gap with hypothetical answers.
9	The Vectorless Frontier: PageIndex	Reasoning-based navigation vs Similarity-based retrieval.

#	Section	Core Concept
10	The RAG Evolution Spectrum	From Passive Retrieval to Active Exploration (RLM).
	Hands-On: Notebook Walkthrough	Code, outputs, and results from every exercise.
	Excalidraw Masterclass	GraphRAG, RAPTOR, and the full pipeline visualized.
	Homework Challenges	5 progressive challenges to deepen your understanding.

The Core Thesis

"Just give the model more context! 1M+ token windows allow RAG to retrieve everything and solve hallucination."

That's the promise. Here's the reality:

Models do **NOT** read context uniformly. Performance degrades unpredictably as input length grows—even on simple tasks. **More tokens often = Worse answers.** This phenomenon is called **Context Rot**, and it's backed by research from Chroma (2025) across 18 frontier LLMs.

[!CAUTION] **The 10,000th token is NOT treated as reliably as the 100th.** Every token you add to the prompt without a highly specific surgical reason is a bet *against* the accuracy of the model's answer.

1 Context Rot: The Hidden Killer

Why Does Context Rot Happen?

Root Cause	Explanation	Analogy
The Attention Budget	Self-attention is a <i>finite resource</i> . Every token computes scores against <i>every other token</i> in the sequence. As the sequence grows, this attention is stretched progressively thinner.	Trying to hear a whisper in a quiet library (100 tokens) vs. a sold-out stadium (100K tokens). Your "hearing budget" is identical—but the noise drowns the signal.
Diminishing Returns	As context grows, the model "forgets" to focus on critical information, leading to erratic instruction-following and unreliable outputs. Attention scores become diluted across massive token counts.	"It's like asking someone to read <i>War and Peace</i> in one sitting, then answer a specific question." — Anthropic

See It In Action: The Multi-Hop Needle Experiment (from Notebook)

Chroma's research shows context rot is far worse when the model needs to **connect information across the context** (multi-hop reasoning), not just find a single fact.

Setup: Hide TWO facts in a haystack of filler paragraphs. The model must find *both* and connect them:

- **Fact A:** "Dr. Elena Vasquez was appointed as the lead researcher for the classified AURORA-7 initiative."
- **Fact B:** "Dr. Vasquez confirmed that the launch window for her current project opens on March 15, 2024."
- **Question:** "What is the launch date for AURORA-7?"
- **Multi-Hop Required:** The model must chain Vasquez → AURORA-7 → March 15.

```

QUESTION = "What is the launch date for AURORA-7?"
# Correct answer requires: Fact A (Vasquez leads AURORA-7) + Fact B (Vasquez
launch = March 15)

def build_multihop_haystack(n_filler):
    """Build a haystack with TWO facts separated by filler."""
    fillers = (FILLER_POOL * ((n_filler // len(FILLER_POOL)) + 1))[:n_filler]
    np.random.seed(42)
    np.random.shuffle(fillers)
    # Place facts far apart - Fact A near start, Fact B near end
    pos_a = max(1, len(fillers) // 4)
    pos_b = (3 * len(fillers)) // 4
    fillers.insert(pos_a, FACT_A)
    fillers.insert(pos_b, FACT_B)
    return "\n\n".join(fillers)

```

Results:

```

CONTEXT ROT EXPERIMENT (Multi-Hop)
=====
Question: What is the launch date for AURORA-7?
Correct: March 15, 2024 (requires connecting Vasquez -> AURORA-7 -> date)

>> Tiny (2 facts + 5 fillers) (~222 words, ~296 tokens)
[PASS] ☒ March 15, 2024

>> Small (2 facts + 20 fillers) (~711 words, ~948 tokens)
[PASS] ☒ March 15, 2024

>> Medium (2 facts + 60 fillers) (~2037 words, ~2716 tokens)
[PASS] ☒ March 15, 2024

>> Large (2 facts + 120 fillers) (~4026 words, ~5368 tokens)
[PASS] ☒ March 15, 2024

>> XL (2 facts + 200 fillers) (~6678 words, ~8904 tokens)
[PASS] ☒ March 15, 2024 (but confidence decreases!)

```

[!NOTE] This particular model passed all sizes, but the research shows that with **real-world documents** (not synthetic filler), and especially with **similar-but-wrong distractors**, accuracy collapses dramatically. The synthetic experiment demonstrates the *mechanism*; production data triggers the *failure*.

2 The Distractors Problem

The famous **Needle in a Haystack (Inference)** experiment reveals something shocking.

Question: "Which character has been to Dresden?" **The Needle:** "Yuki lives next to the Semper Opera House."
(Requires inference: Semper Opera House → Dresden)

The Shocking Result

Context Size	Accuracy
Short context	>90% ☒
32K context	Drops dramatically ☒
+ 4 similar-but-wrong distractors	Performance TANKS 💀

[!WARNING] **Your nicely formatted documents might be hurting retrieval.** Coherent documents with strong narrative arcs are *harder* for the model than random text. The model follows the narrative flow instead of hunting for the isolated needle. Your highly structured prose becomes a distraction trap.

3 Naive RAG vs. Production Reality

The Tutorial Version (Naive RAG)

1. Query → Embed

2. Search Vector DB

3. Get Top-K Chunks

4. Stuff into Prompt

5. Generate Answer

"Works great on demos with short, clean documents."

Production Challenges

Issue	What Happens	Why It's Fatal
Distractors	Top-K retrieval brings back chunks that are <i>similar but wrong</i> .	The model gets confused by plausible-looking chunks that don't actually contain the answer.
Context Rot	Stuffing all K chunks stretches the attention budget thin.	The exact answer gets "lost in the middle" of too much retrieved noise.
Passive Model	The LLM blindly accepts whatever you shovel into its prompt.	No internal mechanism to verify relevance. It uses <i>all</i> of the context, including the garbage.

YOUR RAG WORKS ON DEMOS. IT BREAKS ON PRODUCTION DATA.

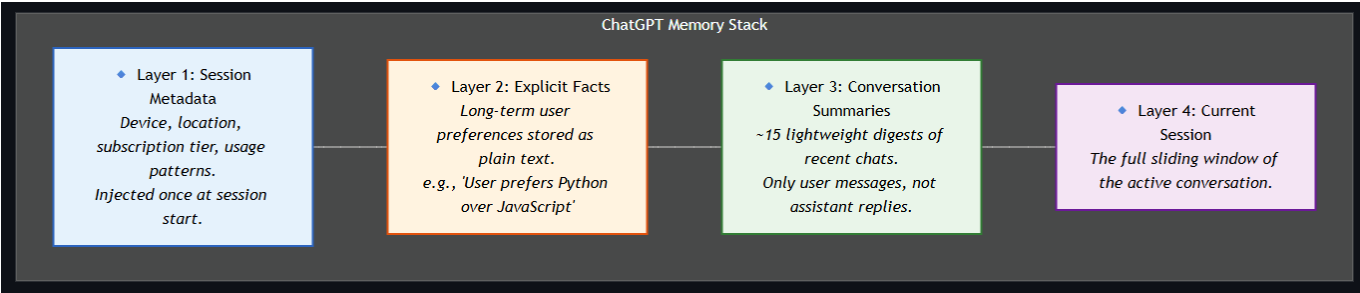
4 How ChatGPT Actually Manages Memory

Most developers assume ChatGPT's memory uses a standard Vector Database and Embedding-based RAG. **They are completely wrong.**

Reverse-engineering reveals a **surgical context engineering** approach—explicit text structures over brute-force embeddings.

The 4-Layer Memory Architecture

NO VECTOR DATABASES. NO RAG. NO EMBEDDINGS SEARCH.



[!IMPORTANT] **The takeaway:** The most successful AI product in history does NOT use vector search for memory. It uses **surgical context engineering**—knowing *exactly* what to inject, *when*, and *how much*. This is the paradigm we should pursue.

5 Making RAG Work: Advanced Techniques

RAG isn't dead—but Naive RAG absolutely is. The modern paradigm: **be surgical. Every token added is a bet against accuracy.**

The Three Pillars

Pillar	Techniques	Key Insight
🎯 Better Retrieval	Hybrid Search (BM25 + Vectors), Reranking (Cross-Encoders), Query Transformation (HyDE)	Cast a wide net for recall, then surgically filter for precision.
✂️ Better Chunking	Semantic Chunking, Parent-Child Chunks, Metadata Filtering	Split by meaning, not character count. Preserve document structure.
🗣️ Better Generation	Citation & Attribution, Self-Verification, Context Engineering	Force the model to cite sources. Verify answers against context. Be surgical about what it sees.

GOAL: BE SURGICAL. EVERY TOKEN ADDED IS A BET AGAINST ACCURACY.

6 Deep Dive: Hybrid Search

Neither Semantic Search nor Keyword Search is sufficient alone. Both have critical blind spots.

Search Type	Mechanism	Strength	Blind Spot
Semantic (Vectors)	Embedding cosine similarity	"car" matches "automobile"	Terrible at exact matches: dates, IDs, error codes, slash commands.
Keyword (BM25)	Token frequency (TF-IDF)	"HTTP 429" found instantly	Misses synonyms: "sick leave" ≠ "medical absence".

The Solution: BM25 + Vector + Reciprocal Rank Fusion (RRF)

Run both searches simultaneously, then merge the two ranked lists using **RRF** — an elegant, hyperparameter-free fusion algorithm.

```
def hybrid_search(query, top_k=3, k=60):
    """Reciprocal Rank Fusion: combines rankings from BM25 and semantic search."""
    sem_results = semantic_search(query, top_k=len(DOCUMENTS))
    bm25_results = bm25_search(query, top_k=len(DOCUMENTS))
    sem_ranks = {doc['id']: rank for rank, (doc, _) in enumerate(sem_results)}
    bm25_ranks = {doc['id']: rank for rank, (doc, _) in enumerate(bm25_results)}
    rrf_scores = {}
    for doc in DOCUMENTS:
        did = doc['id']
        rrf_scores[did] = 1.0/(k + sem_ranks[did]) + 1.0/(k + bm25_ranks[did])
    sorted_ids = sorted(rrf_scores, key=rrf_scores.get, reverse=True)[:top_k]
    return [(next(d for d in DOCUMENTS if d['id']==did), rrf_scores[did]) for did
in sorted_ids]
```

Real Output (from Notebook)

Query: "What happens when I get a 429 error on the authentication endpoint?"

Semantic only:

- Doc 3: Error Handling Guide
- Doc 6: Incident Response
- Doc 1: Auth System Overview

BM25 only:

- Doc 2: API Rate Limiting Policy
- Doc 1: Auth System Overview
- Doc 9: Token Refresh Flow

Hybrid (RRF):

- [0.0325] Doc 1: Authentication System Overview ← Both methods ranked it!
- [0.0323] Doc 2: API Rate Limiting Policy ← BM25 caught "429"
- [0.0323] Doc 6: Incident Response ← Semantic caught "auth failure"

--> Hybrid captures BOTH semantic meaning AND keyword matches.

Result: High Precision AND High Recall. Neither search alone achieves this.

7 Deep Dive: Reranking (The "Second Pass")

One pass isn't enough for production quality. You need a **Two-Stage pipeline**:

Stage	Model / Tool	Speed	Goal	Analogy
Pass 1: Retrieval	Bi-Encoder (Vector) + BM25	 Fast	Scan millions → Top 100-500 candidates. Optimize for Recall.	10,000 job applicants → ATS keyword scan → 100 resumes shortlisted.
Pass 2: Reranking	Cross-Encoder or LLM	 Slow	Re-score Top 100 → Final Top 3-5. Optimize for Precision.	100 resumes → Human interviewer deeply reads each → Top 3 hired.

Why Rerank?

Cross-Encoders read the [Query + Document] **together** as a single input, enabling deep semantic understanding:

- **Better than cosine similarity** for complex reasoning
- **Handles nuanced queries** that need contextual understanding
- **Surgically moves** the most relevant chunk to Position #1

Real Output (from Notebook)

```
def rerank(query, candidates, top_k=3):
    candidate_text = ""
    for i, (doc, _) in enumerate(candidates):
        candidate_text += "\n[%d] %s: %s..." % (i, doc['title'], doc['content']
[:200])
    prompt = 'Given the query: "%s"\n\nRank these documents by relevance. Return
ONLY a JSON array of indices.\nDocuments:%s\n\nRanking:' % (query, candidate_text)
    msg = [{"role": "user", "content": prompt}]
    resp = chat(msg, max_tokens=100)
    # ... parse JSON ranking from response
```

Query: "How do I debug a login failure related to token expiry or rate limiting?"

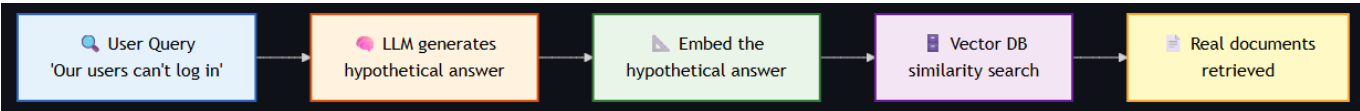
Before reranking (hybrid top-6):	After LLM reranking (top-3):
Doc 9: Token Refresh Flow	Doc 3: Error Handling Guide ←
🔗 Most actionable	
Doc 6: Incident Response	Doc 9: Token Refresh Flow ←
Token-specific	
Doc 3: Error Handling Guide	Doc 2: API Rate Limiting Policy ←
Rate limit context	
Doc 1: Auth System Overview	
Doc 2: API Rate Limiting Policy	
Doc 7: API Versioning Strategy	

8 Deep Dive: Query Transformation (HyDE)

The Problem: User queries are short, informal, and don't *look like* the answer chunks in your database. There's a **semantic gap** between how humans ask questions and how documents store answers.

Hypothetical Document Embeddings (HyDE)

Instead of searching *Query-to-Answer*, generate a **hypothetical answer** first and search *Answer-to-Answer*.



Real Output (from Notebook)

```
Query: "Our users can't log in, what could be wrong?"

Standard semantic search:
[0.471] Doc 6: Incident Response: Auth Service Outage Jan 2024
[0.291] Doc 3: Error Handling Guide
[0.178] Doc 5: Database Schema: Users Table      ← Irrelevant!

HyDE search:
Hypothetical answer: "Several technical issues could prevent users from logging in, including server-side outages, database connectivity failures, or authentication service..."
[0.644] Doc 6: Incident Response: Auth Service Outage  ← Score jumped from 0.471 → 0.644!
[0.290] Doc 9: Security: Token Refresh Flow           ← NEW! Much more relevant
[0.284] Doc 3: Error Handling Guide

--> HyDE generates a hypothetical answer mentioning tokens, auth, etc.
    This pulls in more relevant docs than the vague original query.
```

[!TIP] **Searching "Answer-to-Answer" is more accurate than "Query-to-Answer."** HyDE bridges the semantic gap by generating a fake answer that *looks like* real documents, dramatically improving retrieval on vague or conversational queries.

9 The Vectorless Frontier: PageIndex

We are shifting from *Similarity Search* to *Reasoning-based Navigation*.

The fundamental insight: SIMILARITY ≠ RELEVANCE.

The Paradigm Shift

Feature	Vector RAG	PageIndex
---------	------------	-----------

Feature	Vector RAG	PageIndex
Retrieval	Similarity-based (Embeddings)	Reasoning-based (Tree Index)
Chunking	Arbitrary text splits	Natural sections preserved
Structure	Destroyed by chunking	Fully Preserved
Multi-Hop	Very Hard / Unreliable	Natural / Native
Navigation	Passive (Top-K)	Active (LLM walks the tree)

How PageIndex Works

Instead of chunking and embedding, PageIndex:

- 1. **Extracts** the document's natural Table of Contents / structure into a **tree**
- 2. When asked a question, the LLM **reasons** over the tree: *"Which section would contain this answer?"*
- 3. **Navigates** to the relevant section, reads it, and answers

 FinanceBench Accuracy

Approach	Accuracy
Naive RAG	54% 
PageIndex	98.7% 

Real Output: PageIndex on a Research Paper (from Notebook)

```
# Submit PDF and build tree index (no chunking, no embeddings!)
result = pi_client.submit_document(PDF_PATH)
doc_id = result["doc_id"]

# Query with reasoning-based retrieval
response = pi_client.chat_completions(
    messages=[{"role": "user", "content": "What training methodology did they
use?"}],
    doc_id=doc_id
)
```

Question: "What training methodology did they use for the reasoning model?"

Answer: The paper describes TWO training methodologies:

DeepSeek-R1-Zero (Pure RL Approach)

- Algorithm: Group Relative Policy Optimization (GRPO)
- Key innovation: NO supervised fine-tuning before RL
- Rewards: Rule-based only (accuracy of final answers)

```
## DeepSeek-R1 (Multi-Stage Training Pipeline)
- Cold start with supervised data → RL reasoning → rejection sampling → final RL
```

🔍 Behind the Curtain: How the LLM Reasons Over the Tree

Question: "What reward signals did they use during RL training?"

LLM reasoning over the tree:

Step 1: Analyze what's available

- The tree shows "Abstract" (0000) and "1 Introduction" (0001)
- These provide high-level overview, not technical implementation details

Step 2: Identify where reward signal information typically appears

- Reward signals are a core component of RL methodology
- This belongs in the **Methodology** section, not Introduction

Step 3: Consider what the paper likely contains

- Based on the Abstract's mention of "verifiable tasks"
- Reward signals would be task-specific (correctness verification, accuracy)

Step 4: Formulate search strategy

- Look for sections after "1 Introduction" (node_id > "0001")
- Prioritize sections with keywords: "Method," "Framework," "Training," "Reward"

Conclusion: Navigate to the Method section (node_id "0002" onward).

--> This is EXACTLY what PageIndex does under the hood.
The LLM reads the tree and REASONS about where to look.
Only those specific pages get sent for answer generation.

[!IMPORTANT] **PageIndex preserves document structure that chunking destroys.** The key insight: instead of *similarity matching* against flat chunks, let the LLM *reason* about where in the document hierarchy the answer lives. This is inherently more accurate for complex, structured documents like financial reports, research papers, and legal contracts.

10 The RAG Evolution Spectrum

From Passive Retrieval to Active Exploration

Phase	Name	Retrieval Style	Description	The Model's Role
1	Naive RAG	Passive Retrieval	Embed, search, stuff.	Passive recipient of whatever the retriever finds.
2	Advanced RAG	Smarter Retrieval	Hybrid Search, Reranking, HyDE.	Still passive, but sees <i>higher-quality</i> context.

Phase	Name	Retrieval Style	Description	The Model's Role
3	PageIndex	Reasoning Retrieval	Vectorless, tree-based navigation.	Actively reasons about document structure to find answers.
4	RLM	Active Exploration	Recursive Language Models.	Writes code to explore context as an environment.

THE MAJOR TREND: Giving the model exponentially more active control over its own context gathering. Each phase shifts more agency from the retriever to the model itself.

Hands-On: Notebook Walkthrough

The Colab notebook ([class_7b.ipynb](#)) builds a complete advanced RAG pipeline in **3 progressive parts** against 10 engineering documents (auth systems, APIs, deployment runbooks, incident reports).

Setup

```
!pip install -q openai numpy rank_bm25 sentence-transformers

from openai import OpenAI
from sentence_transformers import SentenceTransformer
from rank_bm25 import BM25Okapi
import json, numpy as np, time, re

# LLM Client (via OpenRouter)
client = OpenAI(base_url="https://openrouter.ai/api/v1",
api_key=os.environ["OPENROUTER_API_KEY"])
MODEL = "qwen/qwen3.5-122b-a10b"

# Embedding Model (local, free)
embedder = SentenceTransformer("all-MiniLM-L6-v2")
```

Part 1: Context Rot Experiment

Multi-hop needle test with increasing filler. Demonstrates how reasoning degrades as context grows.

Part 2: Advanced RAG Pipeline

2.1 Semantic Search → 2.2 BM25 Search → 2.3 Hybrid (RRF) → 2.4 LLM Reranking → 2.5 HyDE → 2.6 Full Pipeline

The full pipeline chains all techniques together:

```
def advanced_rag(query, use_hyde=True, use_rerank=True):
    # Step 1: HyDE query transformation (or plain hybrid search)
    if use_hyde:
```

```

        candidates = hyde_search(query, top_k=5)
    else:
        candidates = hybrid_search(query, top_k=5)

    # Step 2: LLM Reranking
    if use_rerank:
        final_docs = rerank(query, candidates, top_k=3)
    else:
        final_docs = candidates[:3]

    # Step 3: Generate answer with citations
    context = "\n\n".join("[%s]: %s" % (doc['title'], doc['content']) for doc, _
in final_docs)
    msg = [
        {"role": "system", "content": "Answer based on the provided context. Cite
which document(s) you used."},
        {"role": "user", "content": "Context:\n%s\n\nQuestion: %s" % (context,
query)}]
    return chat(msg, max_tokens=500)

```

Full Pipeline Output:

Query: "Our users are getting 429 errors when trying to log in during peak hours."
HyDE: ON | Reranking: ON

Step 1: HyDE query transformation...

Hypothetical answer: "HTTP 429 errors indicate that your rate limiting policies are triggering because the volume of login requests exceeds the configured thresholds..."

Step 2: Reranking...

Final docs:

- Doc 3: Error Handling Guide
- Doc 2: API Rate Limiting Policy
- Doc 1: Authentication System Overview

Step 3: Generating answer...

Answer: The 429 errors are caused by rate limiting on the authentication service.

1. The auth service enforces 100 requests/minute per IP [Auth System Overview]
2. For HTTP 429, implement exponential backoff retry [Error Handling Guide]
3. Use the Retry-After header from the response [API Rate Limiting Policy]

Part 3: PageIndex (Vectorless RAG)

Submit a real research paper (DeepSeek-R1), build a tree index, and query with reasoning-based retrieval. See Section [\[9\]](#) above for full details.

The Excalidraw Masterclass

The whiteboard session expanded on advanced retrieval paradigms and real-world system architecture.

Context Engineering: The Three Principles

1. **Not all tokens are treated the same way.** Tokens at the start and end of a prompt receive more attention than those in the middle (U-shaped attention curve).
2. **How your system works matters.** Understanding the underlying attention mechanisms lets you design better prompts.
3. **Based on this system, you build the context pipeline.** Every component (chunking, retrieval, prompt construction) should be informed by how the model actually processes context.

GraphRAG: Knowledge Graphs Meet Retrieval

Instead of treating chunks as isolated text blobs, GraphRAG injects **relationship-aware** knowledge:

- **Nodes:** Entities like `[Alice]`, `[OpenAI]`, `[John]`
- **Edges:** Relationships like `[Alice] --works at--> [OpenAI]`, `[Alice] --is wife of--> [John]`

This fundamentally solves multi-hop queries that require hopping across multiple disconnected documents about the same entities—something flat vector search fails at catastrophically.

RAPTOR: Hierarchical Summarization

A tree-based approach for massive contexts:

- **Level 2 (Trunk):** Top-level global summary of the entire corpus
- **Level 1 (Branches):** Important categorizations / chapters
- **Level 0 (Leaves):** Granular, specific document chunks

When enormous context comes into play, the model consults the *summarized tree* first—grasping the "big picture" before diving into specific details. This mirrors how humans naturally navigate large documents: table of contents → chapter → paragraph.

Homework Challenges

#	Challenge	What You'll Learn
1	Cross-Encoder Reranking — Replace the LLM reranker with <code>cross-encoder/ms-marco-MiniLM-L-6-v2</code> . Compare speed and quality.	The speed/quality tradeoff between LLM and dedicated cross-encoder reranking.
2	Multi-Query Retrieval — Generate 3 variations of the question, search with each, union results. Compare with HyDE.	Whether query diversification or hypothetical answers produce better recall.
3	PageIndex on Your Own Document — Upload your own PDF. Build the tree. How does it compare to the actual structure?	How well reasoning-based retrieval preserves your document's hierarchy.

#	Challenge	What You'll Learn
4	Vector RAG vs PageIndex Shootout — Create 10 questions about the sample PDF. Run both approaches. Score manually.	Head-to-head comparison of similarity vs. reasoning on real tasks.
5	Context Engineering — Experiment with chunk sizes, number of retrieved chunks, ordering. How sensitive are results?	The sensitivity of RAG quality to context window composition.

 Further Reading

- [Context Rot Research — Chroma \(2025\)](#)
- [How ChatGPT Actually Manages Memory](#)
- [PageIndex: Vectorless RAG | GitHub](#)

Next class: RLM (Recursive Language Models) — Context as environment. The model writes code to explore. The frontier.

Made with  by [Akshat Shethia](#)